

Campus Log Anomaly Detection under Proxy Labels: A TCN–Transformer Study and Design for Local RCA Narration

Thanh Tien Cao^{1*}, Tu Hoang Trinh¹, and Ha Manh Tran¹

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages – Information Technology,
Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, 23dh113972@st.huflit.edu.vn, hatm@huflit.edu.vn

Abstract. Modern campus infrastructures emit heterogeneous logs, but real labels and leakage-resistant temporal partitions are difficult to obtain. We evaluate a masked TCN–Transformer detector on a selective HUFLIT stream of 1,000,000 lines from five backup days. A custom similarity parser produces template IDs; heuristic HTTP-status and keyword rules provide *proxy* labels rather than SOC-adjudicated ground truth. Training replaces 15% of nominal-window tokens with a reserved mask token, and inference uses leave-one-position-out pseudo-log-likelihood. We use a stream-position 80/10/10 split before windowing, five seeds, and $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$. The TCN–Transformer obtains F1 0.0395 ± 0.0163 , ROC-AUC 0.7226 ± 0.0178 , and AUPRC 0.8356 ± 0.0139 ; removing the TCN yields F1 0.3475 ± 0.4097 and ROC-AUC 0.7685 ± 0.0351 , so the experiment does not support a TCN benefit. An audit found repeated records across dated backups (36.2% of unique validation lines also occur in training), so these are descriptive stream results, not leakage-free generalization estimates. We additionally specify a local LLM+RAG narration interface, but the artifact contains only a prompt/schema mock and we report no RCA quality or latency result.

Keywords: AIOps · Log Anomaly Detection · Proxy Labels · TCN–Transformer · Pseudo-Log-Likelihood · Campus Infrastructure.

1 Introduction

Enterprise and higher-education services generate heterogeneous operational logs. Sequence models can prioritize unusual template patterns, but an unusual pattern is not by itself proof of an attack, fault, or root cause [2, 3].

Sequence-based detectors include DeepLog [2], LogAnomaly [3], LogRobust [4], and SwissLog [5]. Published results on curated benchmarks do not remove two practical limitations:

* Corresponding author: thanhct@huflit.edu.vn

1. **Explainability Deficit:** Contemporary anomaly detectors typically expose only a scalar anomaly score or reconstruction error. Such numbers provide little contextual explanation of *why* a sequence is anomalous, *what* attack or fault vector is likely, or *which* services are impacted.
2. **Prolonged Incident Triage & SOC Alert Fatigue:** Security Operations Center (SOC) analysts and network reliability engineers must manually query raw distributed log stores, correlate timestamps across multi-tier appliances, and draft remediation steps. In campus settings this manual triage often dominates mean-time-to-repair for multi-service incidents (credential stuffing, broadcast storms), even when the detector already raised an alert.

To overcome these operational challenges, recent 2025–2026 literature—including LLM4Log (145 studies) [18], KRONE (ICDE 2026) [19], MaidLog [20], Smart Eye (WWW 2026) [23], and cloud RCA recommenders [13, 14]—already explores LLMs for log analytics and incident narration. The remaining gap for campus operators is narrower: (i) measured evaluation on real multi-service campus telemetry (not only public HDFS/BGL), (ii) a lightweight local detector that can gate expensive narration, and (iii) an edge-oriented RCA *design* that avoids cloud APIs. This paper addresses (i)–(ii) with measured experiments and presents (iii) as an architecture whose quantitative RCA study is still pending.

In this paper, we study a campus pipeline combining a custom similarity parser, a masked TCN–Transformer detector, and a designed local 4-bit LLM+RAG narration stage. We do *not* claim a new LLM-for-logs system; the measured contribution is a campus-stream detector study with explicit proxy-label and duplicate-backup limitations.

The core contributions are:

- **Campus-stream evaluation protocol:** Stream-position 80/10/10 partitioning before windowing, nominal-only masked training, $\tau = \mu + 3\sigma$ calibration, five seeds, proxy-label disclosure, and quantified duplicate overlap on a 1,000,000-line HUFLIT selective stream.
- **Masked TCN–Transformer detector:** Real token replacement during training, causal dilated convolutions, positional encoding, attention padding masks, and leave-one-position-out scoring.
- **Same-split baselines:** PCA, DeepLog, LogAnomaly-lite, a masked Transformer without TCN, and the proposed model under identical templates/splits/thresholding (`huflit_baselines.json`).
- **RCA stage as design (unevaluated quantitatively):** A planned local Phi-3/Mistral INT4 + RAG playbook interface; no ROUGE/expert metrics are reported while the harness remains mocked.

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms free-form messages into event templates by separating constants from dynamic parameters [1, 17]. Drain uses a fixed-depth parse

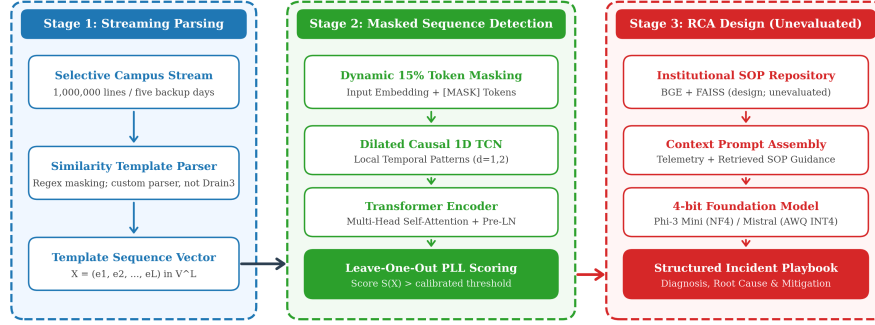


Fig. 1. Architectural overview (Stages 1–2 measured; Stage 3 local LLM+RAG is design-only in this revision).

tree [1]; Spell uses streaming sequence matching [17]. Our artifact does *not* implement Drain or Drain3. It applies regex masking and a linear scan over equal-length templates, merging tokens when positional similarity exceeds 0.5. We call this the *custom similarity parser* and avoid attributing Drain3 properties or throughput to it.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3]. Du et al. [2] introduced DeepLog, employing Long Short-Term Memory (LSTM) networks to model normal log sequences and flag unexpected subsequent templates as anomalous. Meng et al. [3] proposed LogAnomaly, integrating semantic template vectors via FastText to capture semantic similarities. SwissLog [5] and LogRobust [4] further address log instability and interleaved execution threads. While deep sequence architectures achieve strong detection precision, they operate as black-box scoring engines: scalar anomaly metrics fail to explain the operational root cause, imposing heavy manual triage overhead on operators.

2.3 Large Language Models in Log Analytics and RCA

Recent work includes Transformer–TCN detection [24], masked and contrastive representations [22], and LLM-assisted detection such as KRONE [19], Maid-Log [20], CoLA [25], and Smart Eye [23]. LLM4Log [18] surveys this rapidly developing area. LLM-based cloud-incident recommendation has also been studied [13, 14]. Consequently, our architecture is not novel merely for placing a detector before an LLM. We focus on the audited campus-stream experiment; the local narrator remains an unevaluated design.

[†]Legacy batched ms retained in JSON but not relabeled as batch-1 PLL. [‡]LogAnomaly-lite reimplement on our templates, not the authors’ FastText stack. [§]Batch-1 leave-one-out PLL wall clock (CPU, 2 threads).

Table 1. Qualitative comparison of related systems (capabilities as published). Latency is *not* cross-comparable: this work reports batch-1 leave-one-out PLL wall time; DeepLog/LogAnomaly-lite rows use legacy batched scoring (NR for true batch-1).

Approach	Parser	Detector	RCA	Local LLM RCA	CPU ms
DeepLog [2]	Spell	LSTM next-token	Scalar score	×	NR [†]
LogAnomaly [3]	Template+FT	Semantic LSTM	Scalar score	×	NR ^{†,‡}
LogRobust [4]	Drain	Bi-LSTM+Attn	Scalar score	×	NR
SwissLog [5]	Online	Attentive Bi-LSTM	Heuristic loc.	×	NR
KRONE [19]	LLM hierarchy	Hybrid+LLM	Nested LLM	Cloud/API-oriented	NR
This work	Custom similarity	Masked TCN-Transformer	Designed local RAG LLM	Design only	52.92 [§]

3 Methodology

3.1 System Architecture Overview

The studied architecture has three stages: (i) **Stage 1 (Parsing)**: selected syslog messages are tokenized by the custom similarity parser; (ii) **Stage 2 (Masked Sequence Anomaly Detection)**: Sliding template windows are masked, vectorized, and scored by our Masked TCN-Transformer Autoencoder; (iii) **Stage 3 (Designed RCA Narration)**: On alerts, a local 4-bit foundation model *is intended* to retrieve SOP passages and emit a structured JSON playbook; this stage is specified below but not quantitatively evaluated in the current artifact.

3.2 Masked TCN-Transformer Denoising Sequence Model

We formulate the detector as a masked template language model: 15% of input tokens are replaced by a reserved mask token, representations pass through a Temporal Convolutional Network (TCN) [7] and Transformer self-attention [8], and a linear vocabulary head predicts the original identities only at selected positions.

Embedding and Dynamic Masking A sliding window of length $L = 20$ with stride $S = 10$ (as used in the measured HUFLIT runs) extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of the template tokens are randomly replaced with a reserved [MASK] token ($ID = |\mathcal{V}| + 1$), distinguishing masked positions from sequence padding ($ID = 0$). Padding masks are applied across self-attention layers to prevent padded positions from corrupting contextual representations. Input tokens are embedded into a d_{model} -dimensional continuous vector space:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

where $\mathbf{P}_{\text{pos}}$ denotes sinusoidal positional encodings [8]:

$$\mathbf{P}_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \mathbf{P}_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \quad (2)$$

Temporal Convolutional Network (TCN) Layer To capture localized temporal patterns across consecutive log events, the embedded sequence $\mathbf{E}$ passes through a stack of 1D dilated causal convolutional layers [7]. For an input sequence $\mathbf{x} \in \mathbb{R}^L$ and a convolutional filter $f : \{0, \dots, k-1\} \rightarrow \mathbb{R}$, the dilated convolution operation with dilation factor d is defined as:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (3)$$

We configure dilation factors $d \in \{1, 2\}$, enforcing causal convolutions for localized temporal extraction without future feature leakage. Residual connections and layer normalization ensure stable gradient propagation:

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\text{Proj}(\mathbf{E}) + \text{GELU}(\text{CausalTCN}(\mathbf{E}))) \quad (4)$$

Transformer Multi-Head Self-Attention Encoder The output $\mathbf{H}_{\text{TCN}}$ is then fed into a Transformer Encoder block to model long-range inter-template correlations. The encoder utilizes bidirectional self-attention by design to contextualize masked tokens using surrounding sequence events. For queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (5)$$

$$\text{MHA}(\mathbf{H}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (6)$$

$$\text{head}_i = \text{Attention}(\mathbf{H}\mathbf{W}_i^Q, \mathbf{H}\mathbf{W}_i^K, \mathbf{H}\mathbf{W}_i^V) \quad (7)$$

The Transformer block uses pre-layer normalization, residual connections, and a two-layer feed-forward network with GELU activation:

$$\text{FFN}(\mathbf{Z}) = \text{GELU}(\mathbf{Z}\mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2 \quad (8)$$

$$\mathbf{Z} = \mathbf{H} + \text{MHA}(\text{LayerNorm}(\mathbf{H})), \quad \mathbf{H}_{\text{out}} = \mathbf{Z} + \text{FFN}(\text{LayerNorm}(\mathbf{Z})). \quad (9)$$

Masked Reconstruction Objective and Anomaly Scoring A linear classification head projects $\mathbf{H}_{\text{out}}$ back to the template vocabulary space $\mathbb{R}^{|\mathcal{V}|}$, serving as the reconstruction decoder. The model is trained to predict original template identities at masked positions via Cross-Entropy loss:

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \log P(T_i = t_i \mid \mathbf{X}_{\setminus \mathcal{M}}) \quad (10)$$

During inference, we compute pseudo-log-likelihood by making one forward pass per valid position (batched over sequences), masking only that position and excluding padding ($ID = 0$):

$$S(\mathbf{X}) = \frac{1}{L_{\text{valid}}} \sum_{t=1}^{L_{\text{valid}}} -\log P(T_t = t_t \mid \mathbf{X}_{\setminus \{t\}}) \quad (11)$$

A validation-calibrated threshold τ flags anomalous sequences:

$$\tau = \mu_{\text{val}} + 3 \cdot \sigma_{\text{val}} \quad (12)$$

where μ_{val} and σ_{val} are the mean and standard deviation of reconstruction loss over nominal validation sequences.

3.3 RAG-Grounded Foundation Model RCA Narration Engine (Design)

When $S(X) > \tau$, the intended Stage 3 activates a local RAG+LLM narrator. This subsection specifies the design; it is not backed by measured LLM outputs in the current repository.

RAG Knowledge Base & Vector Retrieval (design): The intended path would index institutional runbooks with a dense retriever and FAISS, then prompt a local quantized model for JSON playbooks. We report no SOP count, retrieval metric, RCA quality, or LLM latency: `rca_narration.py` only builds a prompt and returns null-valued schema fields marked `DESIGN_ONLY_NOT_MODEL_OUTPUT`.

Prompt schema (design only): The planned prompt would integrate anomalous templates, infrastructure metadata, and retrieved SOP snippets, requesting JSON fields `severity`, `root_cause_summary`, `technical_diagnosis`, and `remediation_playbook`. Claims about measured NF4/AWQ VRAM (e.g., $7.60 \rightarrow 1.80$ GB) are removed until a non-mock LLM run writes those readings.

4 Experimental Evaluation

4.1 Experimental Setup and Benchmark Datasets

We conduct a measured empirical evaluation on HUFLIT campus logs:

1. **HUFLIT Campus Logs (Measured Stream):** The persisted arrays contain 1,000,000 non-empty lines and 368,940,230 streamed text bytes, capped at 200k lines for each of five backup days and 40k per member. Source shares by line are careerhub 48.60%, aca 26.09%, courses 24.07%, portal 0.31%, and one eligible FortiGate log 0.94%; library/web sources contribute zero lines. FortiGate CSV files and members larger than 500 MiB were excluded. Proxy labels use HTTP 4xx/5xx, deny/block fields, and keyword rules; the line proxy-anomaly rate is 7.96%.
2. **Public Benchmarks (Deferred):** Full re-runs of BGL, HDFS, and UNSW-NB15 under the same hardened harness are left to a follow-up revision so that campus numbers remain strictly measured rather than mixed with prior placeholder tables.

Partitioning and overlap: Records are ordered by backup day, ZIP name, member name, and in-member position, then split 80%/10%/10% before sliding windows ($L = 20$, step 10). This is a *stream-position split*, not an event-time

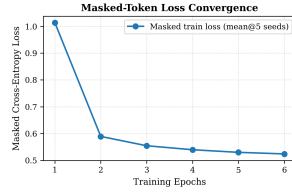


Fig. 2. Masked-token training loss convergence.

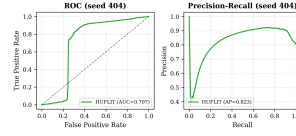


Fig. 3. ROC and Precision-Recall Curves.

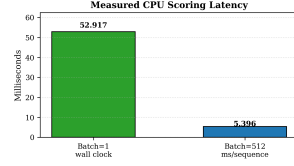


Fig. 4. Measured detector latency (CPU ms/seq).

split. A raw-line hash audit found 34,770 validation-unique lines also in training (36.235% of unique validation lines); validation/test overlap is 8,429 lines (11.382% of unique test lines). Exact sequence overlap is smaller but nonzero: 71 train/validation and 112 validation/test unique sequences. Thus we make no leakage-free claim. Training uses 15% masked token modeling on nominal windows; $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$ is calibrated on nominal validation scores. Seeds are 42, 101, 202, 303, 404.

Hardware & Software Setup: Corrected masked-model runs used CPU only with two PyTorch intra-op threads and one inter-op thread (PyTorch 2.8.0+cpu, Windows 11, Python 3.13). The GPU was not used. Timing is wall clock and includes model forward/scoring overhead.

Table 2. Measured HUFLIT detection (proxy labels; stream-position split; $\tau = \mu + 3\sigma$)

Corpus	Lines	Days	Precision	Recall	F1	ROC-AUC	Batch-1 ms
HUFLIT (selective Thang6 stream)	1,000,000	5	0.3946 ± 0.0315	0.0209 ± 0.0090	0.0395 ± 0.0163	0.7226 ± 0.0178	52.917

4.2 Detection Accuracy vs. Measured Baselines

All detectors use the same persisted stream-position split and custom-parser template IDs, nominal-only training, five seeds, and validation rule. Table 3 is generated from `results/tables/huflit_baselines.json`. Because proxy labels, anomaly prevalence (80.5% in test windows), and duplicate overlap can distort absolute and thresholded metrics, comparisons are descriptive rather than deployment estimates.

Table 3. Measured baselines on the same HUFLIT stream-position split (Mean $\pm$ Std, 5 seeds; $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$)

Model	Precision	Recall	F1	ROC-AUC	Batch-1 ms
PCA (Count Vectors)	0.0000 ± 0.000	0.0000 ± 0.000	0.0000 ± 0.000	0.7379 ± 0.004	NR
DeepLog (LSTM)	0.2598 ± 0.302	0.0057 ± 0.011	0.0110 ± 0.022	0.7808 ± 0.020	NR
LogAnomaly-lite (Emb+LSTM)	0.2716 ± 0.170	0.0054 ± 0.007	0.0105 ± 0.013	0.7772 ± 0.045	NR
Masked Transformer (No TCN)	0.6502 ± 0.272	0.2900 ± 0.362	0.3475 ± 0.410	0.7685 ± 0.035	2.044
Proposed Masked TCN-Transformer	0.3946 ± 0.032	0.0209 ± 0.009	0.0395 ± 0.016	0.7226 ± 0.018	52.917

4.3 Loss Convergence and ROC/PR Curve Analysis

Fig. 2 plots mean masked training loss across five corrected runs. Fig. 3 shows ROC and precision–recall curves for seed 404, with legend values computed from that same seed. Fig. 4 separates measured batch-1 wall time from batch-512 throughput-equivalent time.

4.4 Ablation and RCA Evaluation Status

Table 3 contrasts masked Transformer models with and without the causal TCN. The no-TCN model is better on mean F1 and ROC-AUC; this experiment there-

5 Qualitative Campus Incident Illustrations

The following hypothetical vignettes illustrate the intended schema. They are neither incident findings nor model outputs.

Hypothetical authentication example: Repeated failed-login templates could prompt an analyst to inspect source concentration, affected accounts, and lockout policy; severity and cause require evidence outside the detector score.

Hypothetical network example: Broadcast-related templates could prompt topology and interface-counter checks; a switching loop must not be asserted without corroboration.

6 Discussion & Threats to Validity

Internal & External Validity: Proxy labels, backup duplication, stream-order rather than event-time partitioning, and test anomaly prevalence (80.5%) limit causal and operational interpretation. Gold SOC labels, deduplication before splitting, event-time ordering, and a held-out collection period are required before deployment claims. The custom parser is also unevaluated against parsing ground truth.

Ethics & Data Privacy: Production campus telemetry is kept private. Any future release requires authorization, IP/credential redaction, retention review, and assessment of whether template text can reveal personal or security-sensitive information.

Hallucination Risk: Until measured LLM runs exist, remediation playbooks must not be treated as validated operator guidance.

7 Conclusion

We evaluated a masked TCN-Transformer on a 1,000,000-line selective campus stream and documented a local RCA interface as design only. The present proxy labels, backup duplication, and stream-position split preclude leakage-free or deployment-level claims. Future work must deduplicate before an event-time split, obtain SOC-adjudicated labels, test on a later collection period, and evaluate a non-mock narrator with retrieved evidence.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. In compliance with Springer Nature authorship policies, generative AI tools were used to assist in literature review verification, manuscript refinement, and English copy-editing; all core technical research, architectural designs, empirical implementations, and experimental evaluations were conducted directly by the authors.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach Based on Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019).
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., He, Z., Yu, G.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Dependable Secure Comput. 20(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162857>
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Conf. SysML (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Abidin, M., Aneja, J., Awadalla, H., et al.: Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219 (2024).
10. Jiang, A.Q., Sablayrolles, A., Mensch, A., et al.: Mistral 7B. arXiv preprint arXiv:2310.06825 (2023).
11. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: QLoRA: Efficient Finetuning of Quantized LLMs. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36 (2023).
12. Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S.: AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In: Proc. Mach. Learn. Syst. (MLSys), vol. 6 (2024).
13. Ahmed, T., Ghosh, S., Bansal, C., et al.: Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In: Proc. IEEE/ACM Int. Conf. Softw. Eng. (ICSE), pp. 1737–1749 (2023). <https://doi.org/10.1109/ICSE48619.2023.00149>
14. Soldani, J., Brogi, A.: Anomaly Detection and Failure Root Cause Analysis in (Micro)Service-Based Cloud Applications: A Survey. ACM Comput. Surv. 55(3), 1–39 (2022).
15. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
16. Moustafa, N., Slay, J.: UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. In: Proc. IEEE Mil. Commun. Inf. Syst. Conf. (MilCIS), pp. 1–6 (2015).
17. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
18. Ma, Z., Yang, J., Chen, T.H.: LLM4Log: A Systematic Review of Large Language Model-based Log Analysis. arXiv preprint arXiv:2604.16359 (2026).
19. Ma, L., Liu, J., Zhang, T., VanNostrand, P.M., Hofmann, D.M., Cao, L., Rundensteiner, E.A., Chen, J.: Krone: Hierarchical and Modular Log Anomaly Detection. In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 1773–1787 (2026).
20. Wu, Z., Wang, J., Zheng, L., Zhang, Y., Li, S., Chen, L.: Efficient Zero-shot and Label-free Log Anomaly Detection for Resource-constrained Systems (MaidLog). In: Proc. 42nd IEEE Int. Conf. Data Eng. (ICDE), pp. 671–684 (2026). <https://doi.org/10.1109/ICDE65706.2026.00056>
21. Liu, M., Wang, T., Zhao, Y., Xia, C., Zeng, Y., Tao, Y.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. (2026). <https://doi.org/10.1109/TSC.2026.3700403>
22. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
23. Pei, C., Cui, H., Li, J., et al.: Smart Eye: LLM-Guided Proposer-Verifier Framework for Industrial-Scale Log Anomaly Detection. In: Proc. ACM Web Conf. (WWW), pp. 7869–7879 (2026). <https://doi.org/10.1145/3774904.3792804>
24. Liao, N., Liu, Z.: Log Anomaly Detection Method Based on Transformer and Temporal Convolutional Networks. IEEE Access 13, 68547–68560 (2025). <https://doi.org/10.1109/ACCESS.2025.3561669>
25. Zhu, X., Tang, X., Wu, S., et al.: CoLA: Model Collaboration for Log-based Anomaly Detection. Proc. VLDB Endow. 18(11), 3979–3987 (2025). <https://doi.org/10.14778/3749646.3749668>